A private inbox. Approved campaigns. Useful AI.

Reader edition | Confirmed requirements are distinguished from design proposals and unresolved decisions.

The product

A Windows app for personal inbox management and consent-based campaigns. Connect existing mailboxes, receive notifications, reply, import contacts and history, and prepare campaigns. Approved cloud work continues while the PC is off.

Confirmed pilot	Limit
Private users	10, with completely separate data
Connected mailboxes	Up to 10 per person / 100 total
Outgoing mail	30 per mailbox per day across app paths
Effective maximum	300 per user / 3,000 total per day
Cloud hosting goal	Under A$15/month; exclusions on page 6

Counting each recipient in a rolling 24 hours is proposed, not yet agreed. Aliases and reconnects must share the allowance. Messages sent outside this app cannot be capped by it.

Three separate responsibilities

Hunter / AMF enrich: optional contact and company facts.

AI personalises: Codex, or a restricted Claude fallback, turns permitted facts and your instructions into drafts.

The backend sends: only authorised, approved work that passes consent, suppression and quota checks.

Recommended starting point

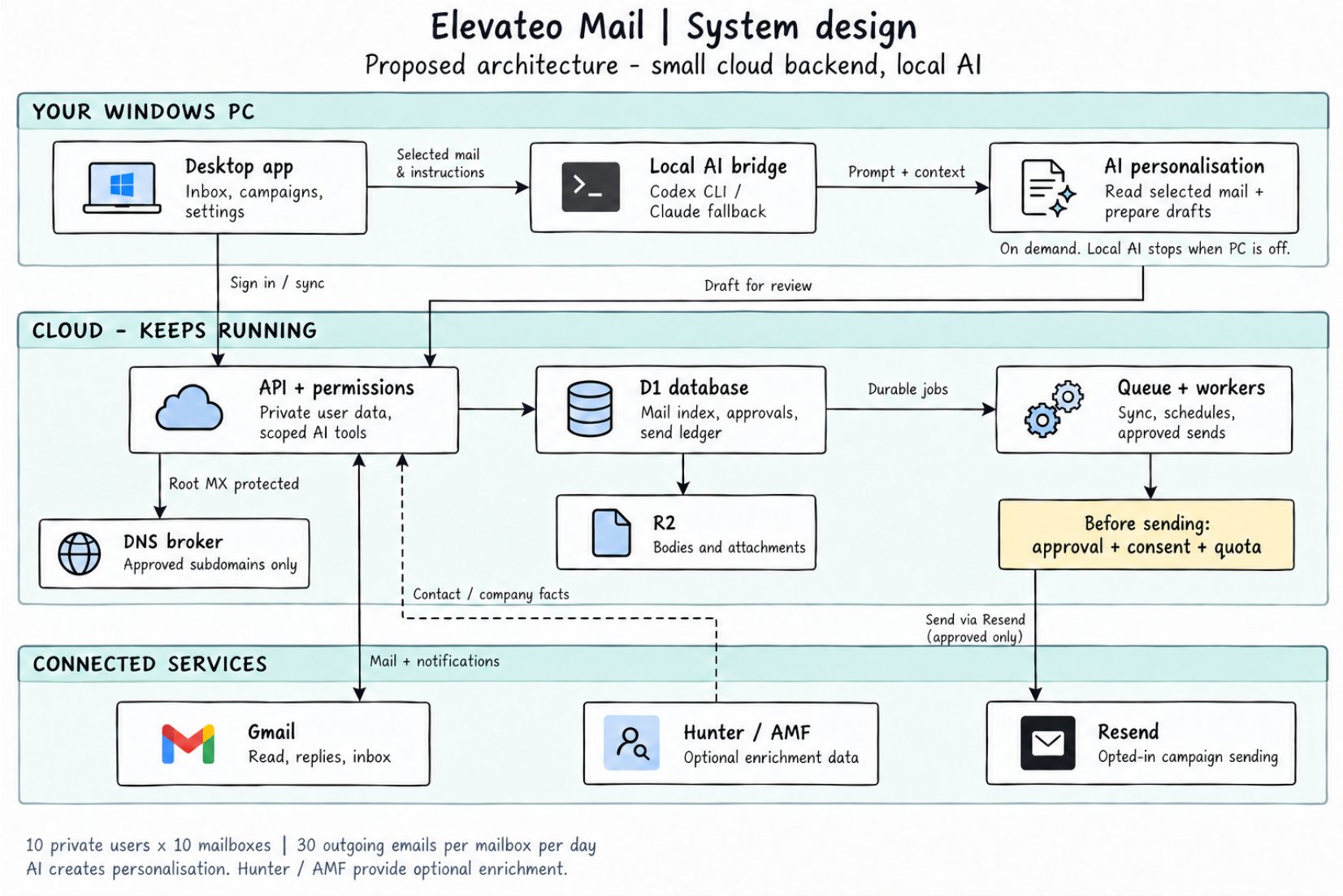
Keep the existing React interface. Add a small Tauri Windows host and one Cloudflare backend using Workers, D1, R2 and Queues. Reuse platform services; add providers only as they are needed.

What exists today

A browser prototype with fictional data. Real mail, authentication, cloud campaigns, AI access and DNS management are still planned. This PDF and its sketches do not claim those features are built.

System architecture

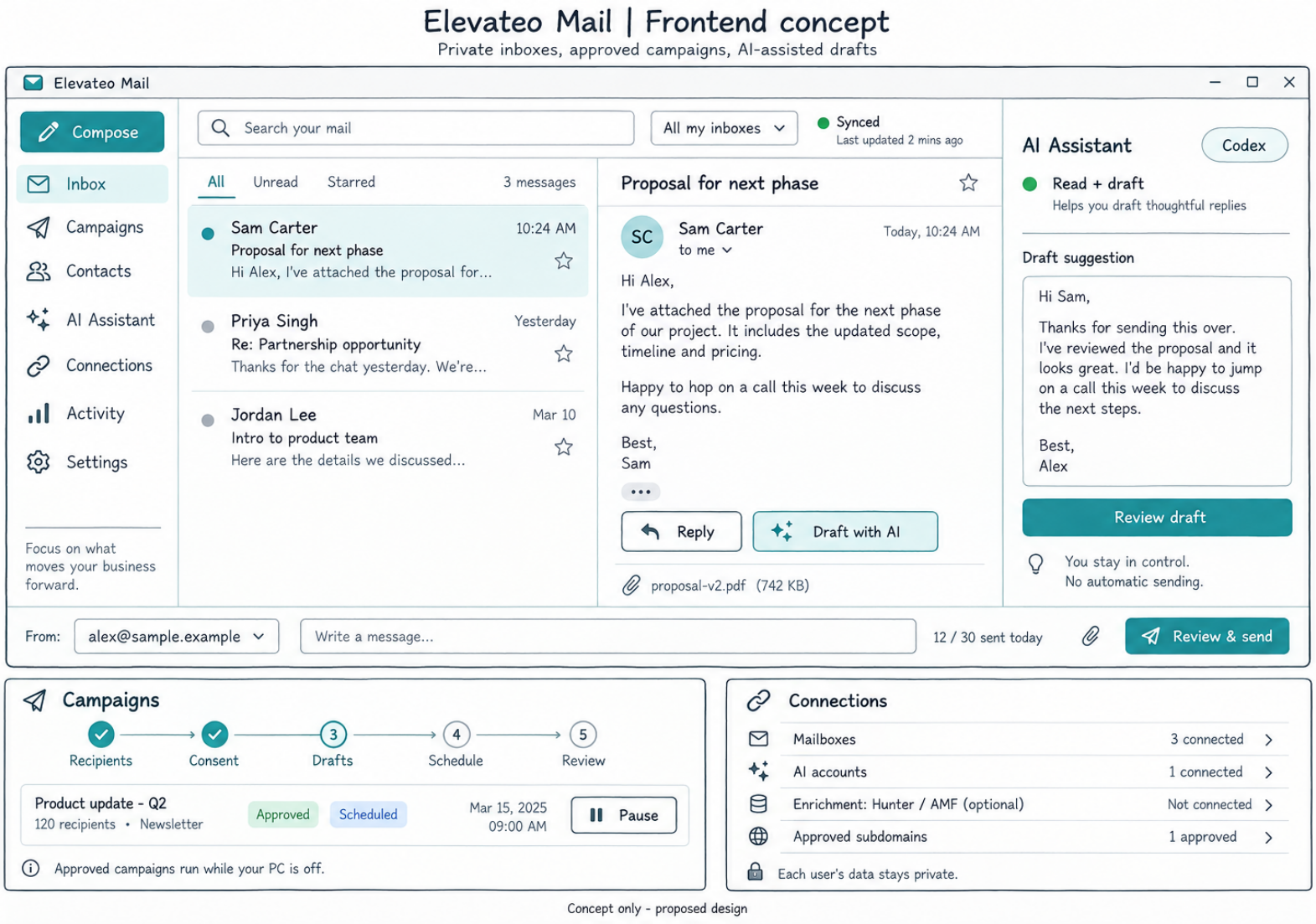
Conceptual responsibilities. Detailed send and recovery rules are on page 4.



All outbound routes, including Gmail, use the same sending checks. Provider events enter an authenticated API and durable processing. D1 stores file references; application code accesses R2. These details are simplified in the sketch.

Frontend sketch

A unified inbox with optional AI help, explicit sending review, and separate Campaigns and Connections areas.



Illustrative content only. Add reconnect, offline, empty, loading and error states; restart-safe drafts; import progress; keyboard access; and clear saved, queued, failed or uncertain send status.

Make email reliable before adding volume

A queue may repeat work. The backend must not turn that into duplicate mail.

Incoming mail

Google authorisation connects an existing Gmail inbox without changing DNS. Save provider IDs and a sync checkpoint. Notifications trigger incremental sync; renew subscriptions and reconcile missed changes. If history expires, perform a bounded resync. Deduplicate events before updating the inbox. [S1]

Outgoing mail

1. Check owner, selected mailbox, sending grant, approved content, purpose and recipient eligibility.
2. Reserve the mailbox allowance atomically and save a send intent plus pending job together.
3. A worker rechecks pause, approval and suppression immediately before dispatch.
4. Record provider evidence and reconcile delivery events.

A timeout is not proof of failure

If a provider may have accepted the message, mark the outcome uncertain and investigate before retrying. Do not promise exactly-once external delivery. Provider acceptance, delivery, bounce and reply are different states. Pausing cannot recall a request already dispatched.

Protect reputation and consent

Resend is for opted-in campaigns from verified domains. It cannot send as a personal @gmail.com address because you do not control gmail.com. Gmail stays the personal inbox; replies return to an authorised receiving mailbox. Address enrichment or validation never proves consent. [S2-S4]

Enforce one set of rules

Keep consent evidence and purpose. Include unsubscribe for campaigns. Stop applicable sends on opt-out, complaint or hard bounce. The 30/day limit covers manual, AI and campaign paths; provider limits may be lower. Reserve room for personal replies. A separate marketing domain can reduce coupling, not guarantee reputation isolation.

Protect existing mail routing

Preserve company root MX records and existing inbound consumers, including the reported in.elevateoco.com setup. A DNS broker may change only explicitly approved subdomain records after an exact preview and human approval. Zone-wide credentials remain a real risk; start read-only or with manual owner changes until containment is agreed.

Enrichment supplies facts. AI writes the draft.

Each user connects their own accounts and keys. Optional services never become a requirement for ordinary inbox work.

The personalisation workflow

Optional enrichment -> permitted facts -> AI draft -> human review -> approved cloud sending.

Hunter and AMF belong only to the enrichment stage. Keep the source and age of their data. Missing facts should be visible, not invented. AI can still draft from user instructions when enrichment is unavailable. The exact AMF product/API remains to be identified.

Connections and permissions

Separate Mailboxes, Sending services, AI access, Enrichment and Domains. Show verified capabilities, last sync, remaining allowance, reconnect and revoke. Enter secrets in secure settings, never AI chat. Encrypt cloud credentials; retain local CLI login material locally. Apply ownership checks to every request and file lookup.

Provider rollout

Gmail first, then Resend. Test one named self-hosted API before promising broad compatibility; it must be reachable when the PC is off. EmailGuard is optional validation/deliverability support. Instantly or Smartlead can follow later, with one scheduling authority per campaign.

Keep AI access narrow

Default to reading selected mailboxes and drafting. Sending is separately enabled and approved. MCP is a tool interface, not a permission system: every tool calls the same protected backend rules. Email bodies, attachments and enrichment are untrusted content, never instructions that grant access. [R2]

Keep the desktop light

Run one CLI task on demand, with bounded input/output, cancellation and a deadline. Stop child processes after use. No local model inference or constant AI polling. Precompute and approve campaign drafts before shutdown; cloud campaigns pause a step if an approved draft is missing.

Sandbox and data-policy gates

Tauri permissions do not sandbox a spawned CLI. Verify Codex isolation on the pinned Windows version; start Claude fallback with tools disabled until its isolation is proven. Never widen privileges on fallback. Local CLI use can send content to AI providers: resolve Google data rules and subscription terms before enabling mailbox context. [S5-S7]

Keep costs low by bounding the work

These are planning scenarios, not a live quote, measured bill or spending guarantee. Pricing references: S8-S11.

Illustrative monthly hosting	Assumption	Estimate
Workers + R2 base	US$5 Workers + US$0.60 storage; proposed 50 GB files including backups	US$5.60
Converted base	Assumed A$1.70/US$1 and illustrative 10% tax	A$10.47
Busy pilot queue scenario	100 mailboxes checked every 5 minutes + 3,000 sends/day; 30 days	+ about A$1.39
Scenario total	Before notification-driven work, retries and other overages	About A$11.87

Control the biggest costs first

Use one backend, not a service fleet. Load metadata before bodies; fetch one page at a time. Index actual queries, batch imports, coalesce notifications, and keep queue payloads small. Use fair per-user concurrency and backoff. Delay optional enrichment and analytics before critical receipts or recovery.

A durable work queue is enough. Redis, Kafka and RabbitMQ have no demonstrated need at this pilot size. D1 remains conditional on storage, search and write measurements; migrate deliberately if it no longer fits. [S9-S10]

Measure these targets

Proposed desktop targets: average idle CPU below 1%; app/WebView2 memory below 250 MB visible or 150 MB in tray; interaction feedback within 100 ms. Measure CLI children separately and together. These are unproven targets on your hardware, not guarantees; normal rendering can use the GPU.

Excluded: mailboxes, Resend/enrichment/AI plans, domains, signing, Google assessment, and any extra independent backup or alerting cost. Storage/history allowances are not agreed. Billing alerts are not hard caps. Recheck shared account allowances and prices before launch.

Professional structure, without empty scaffolding

System design assigns jobs and boundaries. Backend architecture puts those rules into maintainable server code.

Proposed repository destination

Location	Responsibility
apps/desktop/	React interface and minimal Tauri host
apps/backend/	API, scheduler and queue handlers
packages/domain/	Mail, quota, consent and campaign rules
packages/contracts/	Shared validated requests and responses
packages/providers/	Gmail, Resend and optional adapters
infra/	Cloud configuration and migrations
tests/	Provider fakes, integration, GUI and load
docs/ and .planning/	Decisions, research, standards and state

Migrate the working prototype deliberately. Create shared packages only when a second consumer needs them. This is an ownership map, not a request to create empty folders now.

Learn by tracing one action

Action: pause a campaign.
Rule: stop future dispatch; explain work already in flight.
Contract: return saved status and version.
Data: persist the status and send history.
Failure: on connection loss, reload saved state.
Proof: test pause during dispatch and reconnect.

Repeat this six-part exercise for each feature before selecting more services. [S12]

Code rules that earn their keep

Use strict types, boundary validation, one typed configuration module and explicit errors. Keep secrets out of logs and bundles. UI code calls backend use cases instead of duplicating sending rules. Keep migrations and decisions in version control. Comments explain the non-obvious constraints.

Repository identity

Local folder: Code and projects / Elevateo Mail.
Private GitHub: tanzeel-roseyco/elevateo-mail.

The existing prototype stays functional until the planned migration is approved.

Test the failures that could hurt the product

Aim for zero incremental test-service spend with existing resources, not a claim that hardware or licences are free.

Focused automated checks

Run lint, typechecks and meaningful unit/integration tests in Docker. Use fake Gmail, Resend and enrichment adapters, deterministic data and clocks, and existing Vitest/Playwright tooling. Avoid paid API calls in routine tests. Test behaviour and boundaries, not trivial implementation details.

The essential cases

Privacy: one user cannot read another user's mail, keys or files.
Sending: concurrent requests cannot exceed 30; duplicate jobs do not duplicate sends; uncertain outcomes do not trigger blind retries.
Campaigns: pause, unsubscribe and expired approval stop future dispatch.
Recovery: token expiry, lost notifications, provider outages, restore and reconnect.
AI/DNS: hostile emails cannot grant access, exfiltrate secrets or change root MX.

GUI and real Windows checks

Test connect/reconnect, compose, review/send, import, pause and error states. Visually inspect the actual app. Windows install, tray and notifications need native Windows checks; Linux containers cannot reproduce them. Add an authorised live-account smoke test before launch because fakes cannot prove provider compatibility.

Our own small health screen

Show last sync, overdue work, uncertain sends, auth failures, provider errors, quota use, storage and backup age. Use structured redacted logs, short retention and correlation IDs. Do not build a monitoring cluster for ten users; reconsider Sentry only if diagnosing crashes becomes costly.

Alerts must work when the app does not

Use an independent heartbeat checker and a chosen alert destination. Deduplicate alerts, require meaningful thresholds and link each alert to a recovery action. Test backup restoration and key recovery. Backups in the same account are not independent protection. [R3]

What the research changes in practice

Delivery research supports durable records and explicit recovery, not a promise of exactly-once email. Agent research supports treating mail as untrusted and keeping permissions outside AI. Test research supports controlled clocks and targeted fault injection. These are design applications, not proof our implementation works. [R1-R4]

Build in small, provable steps

Review the open sections below before backend implementation. Already-confirmed user and mailbox counts do not need reopening.

Delivery sequence

1. Agree the foundations. Settle external Google approval, data policy, storage allowances and backend standards.

2. Connect one mailbox. Prove sign-in, incremental sync, reconnect and two-user isolation.

3. Prove safe sending. Implement durable intents, quota reservation, consent, suppression and uncertain-outcome recovery.

4. Run approved campaigns. Verify pause, scheduling and PC-off operation; add Resend.

5. Add the Windows AI experience. Prove notifications, resource limits, selected-mailbox grants and constrained fallback.

6. Measure the full pilot. Run ten-user load, restore and monthly-cost checks. Expand provider support only after those pass.

External Google use requires a real verification/assessment plan. Ten users alone do not establish an exemption; testing-mode authorisations are unsuitable for reliable unattended production. Cost and timeline remain launch decisions. [S5]

Review section	Decision still needed
Daily allowance	Rolling 24 hours? Count To, CC and BCC recipients individually?
Data and imports	History age, batch limits, attachments, retention, regions and export/deletion.
Identity and policy	App sign-in/MFA, Google approval and permitted mail-to-AI data use.
Connections	Exact AMF API and first reachable self-hosted mail server.
Domains	Approved subdomains, record types, owner and credential boundary.
Operations	Independent backup, recovery owner, alert destination and costs.
Engineering	Section-by-section approval of draft backend/desktop standards.

Consent purpose and evidence, approved sending identities and recipient jurisdictions must be defined before real campaigns. The plan is specific about what remains unknown; it does not pretend every launch decision is settled.

Sources and implementation handoff

Selected primary references from the completed research. Provider terms and prices must be rechecked at integration.

Provider and design references

S1 Google: synchronising a Gmail client

S2 Resend: acceptable use

S3 Resend: verified sending domains

S4 Google: email sender guidelines

S5 Google: restricted-scope verification

S6 Codex: Windows sandbox

S7 Claude Code: sandboxing

S8 Cloudflare: Workers pricing

S9 Cloudflare: D1 pricing and allowances

S10 Cloudflare: Queues pricing

S11 Cloudflare: R2 pricing

S12 C4: drawing systems at different levels

Research principles applied

R1 Delivery, consistency, and determinism
Separate saved state, repeatable jobs and external side effects.

R2 AgentDojo
Test hostile email and tool misuse; enforce permissions outside AI.

R3 Alert Anti-patterns in Industrial Cloud Systems
Make alerts actionable, bounded and deduplicated.

R4 Feedback-based Automated Failure Testing
Target failures around save, send and acknowledgement boundaries.

For the implementation agent

Use this PDF for product direction, then the repository's backend blueprint, connection contracts and approved standards for exact rules. Historical research is supporting evidence and may use older capacity assumptions. The latest user decisions take precedence.

Six initial research assignments and ten follow-up assignments informed the plan. This edition corrects enrichment versus personalisation and adds two fresh concept sketches. Backend implementation remains unapproved.